FLOWFORGE

The complete viva master guide

From your first web concepts to the engine's design, source behavior, and difficult follow-up questions.

BEGINNER FRIENDLY / SOURCE VERIFIED / 100 VIVA QUESTIONS

28 chapters covering architecture, Python, OOP, APIs, databases, authentication, React, TypeScript, integrations, security, tests, demonstrations, and rapid revision.

Read alongside the separate line-by-line source companion. Implementation facts, planned capabilities, test evidence, and known limitations are distinguished throughout.

Prepared from the repository snapshot of 31 October 2026.
No production deployment verification is claimed.

Contents

1. How to study this guide 4

2. What the project is, in ordinary language 5

3. The actual implementation: facts you must get right 6

4. Web development from zero 7

5. The architecture, and why the layers exist 8

6. Python syntax and language ideas used in this project 9

7. The object-oriented core, with actual code 11

8. Events, context, triggers, and conditions in detail 13

9. The engine, statement by statement 14

10. Database models and the meaning of 'model' 15

11. Sessions, transactions, schemas, services, and repositories 17

12. Every HTTP endpoint and its behavior 18

13. Trace one complete manual workflow 20

14. Generic HTTP actions, network requests, and SSRF 21

15. GitHub: incoming webhook and outgoing dispatch 22

16. Email: Resend and Gmail are different paths 23

17. Authentication, authorization, and browser security 25

18. React, TypeScript, and JSX explained from the source 26

19. The frontend API wrapper and CSS 29

20. Libraries, tools, versions, and alternatives 30

21. Tests: what they prove and what they do not 32

22. Run commands and explain every part 34

23. Known limitations and how to defend them intelligently 35

24. A practical viva demonstration script 36

25. Viva question bank: concise answers with follow-up depth 37

26. Rapid revision: the facts to recall before entering 42

27. Exercises that prove you understand the code 43

28. Source map, report scope, and further reading 44

Prepared from the repository on 31 October 2026. This is a study report about the implementation inspected on that date, not a claim that the production deployment has been verified. Read it beside `FlowForge-Line-by-Line.md`, which reproduces and explains the source with original line numbers.

1. How to study this guide

You do not have to memorize every punctuation mark before you understand the project. Work in this order: understand the problem; trace one successful workflow; understand the objects; trace the API and database; understand the UI; then study integrations, security, tests, and limitations. Finally, use the question bank to practise explaining without reading.

If you have one evening, concentrate on chapters 2-7, 10-13, 17-20, and 24-27. If you have several days, read all chapters and use the line companion to look up every unfamiliar statement. For each function, answer four questions aloud: who calls it, what enters it, what it changes, and what it returns or raises.

In this guide, **implemented** means the relevant code exists; **tested** means a specified automated check exercised something; **planned** means it is in a design document or suggested improvement. Those words are deliberately different. A passing mocked email test does not prove that a real recipient received mail.

Do not invent the expansion of PLP if your department uses a particular course title. Use your department's official expansion. The material here emphasizes programming language concepts and object-oriented design as well as web development.

2. What the project is, in ordinary language

FlowForge is a small workflow automation platform. A workflow is a reusable instruction of the form: **when an event occurs, if a rule is satisfied, perform an action**. Instead of writing a separate program every time you want two tools to work together, you save a configuration and let a common execution engine interpret it.

An everyday analogy is a doorbell system. A button press is the trigger, 'only during office hours' is a condition, and ringing a bell is an action. FlowForge replaces the button with a manual request or GitHub webhook, the office-hours rule with conditions over event fields, and the bell with an HTTP request, GitHub dispatch, or email.

The project has two connected purposes. The product purpose is reusable automation with visible execution history. The academic purpose is to demonstrate that abstraction, inheritance, polymorphism, composition, factories, and composite conditions solve an actual extensibility problem.

A 30-second answer

"FlowForge is a web-based workflow automation platform built around triggers, conditions, and actions. A React and TypeScript frontend lets a user configure workflows. A FastAPI backend saves their configuration, creates Python runtime objects through factories, evaluates the rules, executes actions, and records the result. The core engine calls abstract interfaces, so adding an integration does not require rewriting the engine."

A two-minute answer

"The problem is that developers repeatedly perform tasks between tools, such as reacting to a GitHub event or sending a notification. FlowForge expresses those tasks as configuration. Users sign in through Supabase, create a workflow, choose a manual or GitHub webhook trigger, optionally add a condition, and configure an action. The backend validates and stores the configuration using SQLAlchemy. At execution time, factories hydrate concrete Trigger, Condition, and Action objects. WorkflowEngine checks the trigger, evaluates the condition, and executes the actions sequentially. It returns SUCCESS, FAILED, or SKIPPED, and the service persists an execution record. The implementation includes generic HTTP requests, GitHub repository dispatch, and email through either Resend or a user's OAuth-connected Gmail account. It is a limited MVP: scheduling is not automatic, execution is synchronous, and production hardening remains."

What makes it different from a hardcoded script?

A script usually embeds its conditions and network calls directly in source. FlowForge stores those choices as data and reuses the same orchestration algorithm for different combinations. A script can still be the right solution for one task; FlowForge earns its abstraction when many similar automations need a common interface, ownership, storage, and history.

Do not claim to have matched mature automation platforms in scale, marketplace size, or reliability. The project's strength is a comprehensible, extensible implementation of the essential pipeline.

3. The actual implementation: facts you must get right

Topic	What the current code actually contains
Frontend	React function component, TypeScript, Vite, Supabase JS client, native fetch
Styling	Plain CSS imported by `main.tsx`; Tailwind is a design-document choice, not an installed implementation dependency
Backend	FastAPI, synchronous SQLAlchemy sessions, Pydantic, HTTPX, Python classes
Authentication	Supabase identity lookup when backend `supabase_url` is configured; deliberate unauthenticated local mode otherwise
Workflow UI	Manual and GitHub webhook triggers; zero or one visible condition; one visible action
Backend capabilities	Multiple actions, multiple top-level conditions joined with AND, nested AND/OR condition configuration, comparison conditions
Scheduling	`ScheduledTrigger` checks an event type; no cron process, timer, or scheduler produces those events
Execution	Actions execute during the request; there is no queue, worker, or FastAPI BackgroundTasks usage
Runtime statuses	SUCCESS, FAILED, SKIPPED; a database default of PENDING does not implement a PENDING/RUNNING state machine
Runtime integration abstraction	Trigger, Condition, Action are ABCs; no separate runtime Integration ABC exists
Database integration	An `Integration` ORM model exists, but no general database action or integration marketplace is implemented
Email	Provider selected globally by backend settings: Resend or Gmail; Gmail connections are per user
AI	No language model or machine learning model is part of the product runtime
Deployment	Repository has a deployment runbook; historical production issues in MEMORY are not evidence of today's deployed state

The older phase explanations describe earlier snapshots. For example, Phase 2 says authentication and network actions are deferred; the current source implements them. Explain the current source, and use the older reports only to discuss development progression.

4. Web development from zero

Browser, frontend, backend, and database

The **browser** runs on the user's computer. It downloads HTML, CSS, and JavaScript. HTML describes structure, CSS controls appearance, and JavaScript handles interactions. TypeScript is a development language that adds type checking to JavaScript; the browser executes the generated JavaScript rather than TypeScript annotations.

The **frontend** is the browser-facing part of FlowForge. It displays forms, keeps temporary input state, and asks the backend for data. It should not hold the Google client secret or perform trusted workflow execution.

The **backend** is a Python process on a server. It receives HTTP requests, checks identity and ownership, validates data, loads workflows, executes actions, and talks to the database or external APIs.

The **database** retains information beyond one request or process restart. React state disappears when the page is refreshed; a properly configured persistent database does not. Local SQLite is a development convenience. Production is intended to use PostgreSQL through Supabase.

HTTP and an API

HTTP is a request-response protocol. A request has a method, URL, headers, and sometimes a body. A response has a status code, headers, and sometimes a body. An API is an agreed interface through which programs communicate; FlowForge's API uses HTTP and JSON.

Consider `POST /api/workflows`. POST is the operation, `/api/workflows` identifies the resource collection, `Authorization` carries identity proof, and a JSON body describes the new workflow. The successful response is HTTP 201 with a workflow summary. The frontend uses that response to update its display.

An **origin** is scheme, hostname, and port: `http://localhost:5173` and `http://localhost:8000` are different origins because their ports differ. A **path parameter**, such as `{workflow_id}`, selects a particular resource. A **query parameter**, such as `?user_id=...`, supplies additional request data. A **header** carries metadata, such as a bearer token or webhook signature.

JSON is data, not Python code

JSON supports objects, arrays, strings, numbers, booleans, and null. It does not directly store Python methods, class definitions, datetime objects, or database connections. JavaScript uses `JSON.stringify` to serialize a value and `JSON.parse` to parse a JSON string. FastAPI/Pydantic and SQLAlchemy handle related conversions on the server.

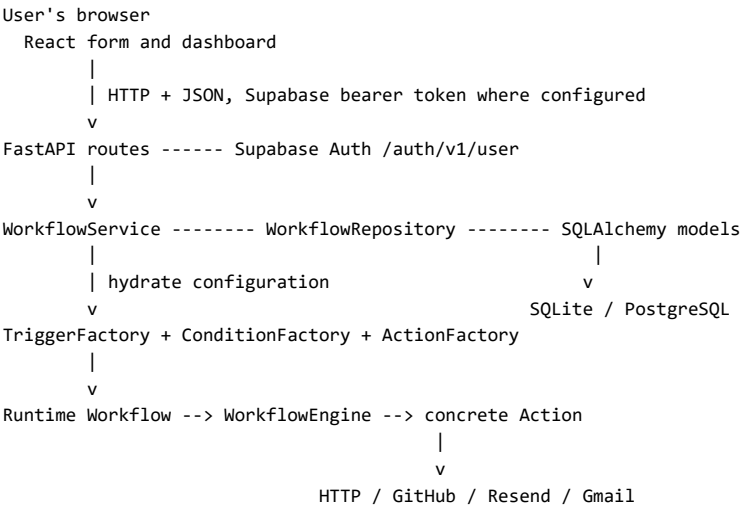
Python spells boolean values `True`/`False` and absence `None`; JSON spells them `true`/`false` and `null`. JavaScript also has `undefined`, which is not a JSON value. A quoted number such as `"3"` is a string, not the number `3`.

Ports, local development, and HTTPS

Port 5173 is the normal Vite development port; port 8000 is the normal Uvicorn port. `localhost` refers to the same machine making the request. A public GitHub service cannot deliver to your computer's private `localhost`; use an appropriately exposed development endpoint or a deployed backend for a live webhook demonstration.

HTTPS adds transport encryption and server authentication through TLS. It protects traffic in transit; it does not repair missing ownership checks or automatically encrypt a token column in the database.

5. The architecture, and why the layers exist



The **route layer** translates HTTP into application calls. The **service layer** coordinates a use case, such as creating or running a workflow. The **repository** groups recurring database access. **ORM models** describe persistence. **Pydantic schemas** describe accepted and returned data. The **core** describes workflow behavior.

Why not place everything in a route? A large route would couple HTTP, database queries, rule evaluation, and provider calls. You could not easily test the engine without a web server. Keeping the engine small lets tests instantiate objects directly.

This separation is a design goal with practical exceptions. Current workflow routes still perform some database lookups and user synchronization, and webhook routes query candidate workflows directly. Concrete actions import settings and GmailService, so the entire `core` directory is not perfectly independent of infrastructure. The engine algorithm itself avoids FastAPI and SQLAlchemy imports. State that narrower, verifiable claim.

This is a layered application, not a microservices deployment. Frontend/backend separation does not make every Python module a separately deployed service. A service class is an application object; a deployed service is a running process/container.

File-reading order

Read `core/event.py`, the three `base.py` contracts, `core/workflow.py`, `core/engine.py`, and `core/factories.py` first. Then read `models/entities.py`, `schemas/workflows.py`, `services/workflow.py`, and workflow routes. Finish with authentication, actions, Gmail, and the frontend. The line companion follows these concepts while retaining exact source references.

6. Python syntax and language ideas used in this project

Imports, modules, and packages

`from app.core.event import Event` imports the name Event from a module. A `.py` file is a module. Directories with `__init__.py` organize packages; these initializer files also re-export selected public names. `__all__` documents names exported by wildcard imports, but it is not an access-control boundary. Importing a module executes its top-level statements, which matters because `settings = Settings()` and database engine creation happen at import time.

`from app.core.workflow import Workflow as RuntimeWorkflow` creates an alias. There are two distinct Workflow classes: a persisted ORM model and a runtime composition object. The alias prevents confusing their roles. Import aliases do not clone a class.

Class, object, constructor, method, and self

A class is a definition of a kind of object. `EqualsCondition` is a class; `EqualsCondition("repo", "FlowForge")` constructs an instance. `__init__` initializes the instance after Python creates it. `self` refers to that particular instance and is passed automatically for an ordinary bound method call. `self.field = field` stores the constructor argument as an attribute.

`condition.evaluate(context)` looks up the instance method, binds self to condition, and passes context. Two EqualsCondition instances can have different expected values because their attributes belong to separate objects.

Type hints do not perform general runtime validation

`def evaluate(self, context: ExecutionContext) -> bool:` says the parameter should be an ExecutionContext and the result should be boolean. Python does not automatically enforce all ordinary annotations. Pydantic models perform validation when models are constructed; `isinstance` checks in action constructors perform explicit runtime checks. Do not confuse those mechanisms.

`str | None` means a string or absence. `list[Action]` means a list whose intended elements implement Action. `dict[str, Any]` means string keys and broadly typed values. `Mapping` describes a mapping interface suitable for read-oriented access. `Iterable` accepts values that can be iterated, not only lists. `tuple[Workflow, RuntimeWorkflow]` describes a fixed two-item return.

Decorators

A decorator transforms or registers a definition. `@abstractmethod` marks a required method for ABC instantiation rules. `@staticmethod` creates a method that does not receive self; the factories use this because object construction does not need factory-instance state. `@dataclass` generates convenience methods such as initialization and representation. `@router.post(...)` registers a path handler. `@pytest.fixture` registers reusable test setup.

These decorators have different jobs despite sharing the same @ syntax. Do not describe every decorator as inheritance or as a function being executed on every request.

Dataclasses and mutable defaults

Event and ExecutionContext are dataclasses. `field(default_factory=dict)` calls dict separately for each new instance. A shared mutable default would let unrelated objects accidentally reuse the same dictionary. The timestamp default is a lambda: it defers `datetime.now(timezone.utc)` until each Event is constructed.

`@dataclass(frozen=True)` on EngineResult and AuthenticatedUser prevents normal reassignment of fields. It is shallow immutability, not proof that every object referenced by a field is immutable. An ORM model serves a different purpose and is not interchangeable with a dataclass.

Expressions and control flow

== compares values; = assigns; `is None` tests identity with the singleton None. and and or short-circuit. `if not ...` rejects a false-like value. `return` ends the function. `raise` exits through an exception. `continue` moves to the next loop iteration; `break` leaves the loop.

`[factory(item) for item in items]` is a list comprehension. `enumerate(items)` produces position/value pairs, allowing action order to be saved. `all(predicate(x) for x in items)` requires every predicate and stops at the first false result. `any(...)` stops at the first true result. Empty `all` is True; empty `any` is False. Consequently an empty AND group passes and an empty OR group fails in the current implementation.

`mapping.get(key, default)` returns a default when a key is absent. `setdefault("action_results", [])` creates the list if necessary and returns the existing or new list. Appending changes that shared execution context's data. An f-string such as `f"Unsupported action type: {action_type}"` interpolates an expression into text.

Exceptions and cleanup

`try/except` handles a failure. `finally` runs cleanup even if the body raises. `raise ... from exc` chains causes; `from None` suppresses the displayed chain. A `with httpx.Client(...) as client` context manager closes resources on exit. A generator dependency uses `yield` to give FastAPI a session, then continues with commit/rollback/close during dependency cleanup.

`async def` defines a coroutine. `await request.body()` allows waiting for request bytes without blocking that part of the event loop. Merely writing async does not make synchronous SQLAlchemy or HTTPX calls nonblocking. FlowForge's webhook handler subsequently calls synchronous execution code; this remains a concurrency limitation.

The Python ABC mechanism is documented in Python's abc reference. The examples in this guide are explanations of FlowForge's use of it, not a claim that Python works exactly like Java or C++ interfaces.

Scope, references, mutability, and binding: useful PLP follow-ups

A local name inside a function normally belongs to that call's local scope. Module-level settings and the FastAPI app belong to module scope. Python resolves ordinary names through local, enclosing, global, and built-in scopes; attributes such as self.field are looked up on objects instead. Passing an object passes a reference to that object through a new local parameter binding, often called call by sharing. Mutating context.data is visible to other code holding that same context; rebinding a local parameter does not rebind the caller's variable.

Dictionaries and lists are mutable. Strings, integers, and ordinary tuples are immutable in their own structure, although a tuple can refer to a mutable object. `self.calls.append(context)` mutates a list rather than replacing the context. The same context is intentionally shared among actions in a run, while default_factory prevents accidental sharing between unrelated runs.

Python is dynamically typed: a runtime object's type determines supported operations. TypeScript provides static checking before its annotations are erased. Python's class method dispatch is late-bound according to the receiver. Overriding means a subclass implements a base method; it differs from Java/C++-style overload resolution among several same-named signatures. Python's multiple inheritance uses a method resolution order (MRO); the timestamp mixin and declarative base are combined according to that order. The project does not implement operator overloading, a custom metaclass, or a custom memory allocator merely because these are language features.

An anonymous lambda is a function expression. A callback is a function supplied to another operation to be called later. A closure can retain access to variables from the scope where the callback was created. React event/effect callbacks therefore see values from a particular render; this helps explain why an update setter does not rewrite an already running callback's captured values.

7. The object-oriented core, with actual code

Abstraction: specifying a capability

```
class Action(ABC):
    @abstractmethod
    def execute(self, context: ExecutionContext) -> None:
        """Perform the side effect."""
```

The contract says an action can execute with an execution context. It hides whether the implementation sends email, invokes an HTTP endpoint, or records a test call. `ABC` supplies abstract-class behavior; `@abstractmethod` means a subclass must supply the method before it can normally be instantiated. A direct `Action()` fails. The return annotation is None because action implementations communicate side effects or mutate context rather than returning the engine result.

Trigger similarly exposes `should_execute(event) -> bool`, and Condition exposes `evaluate(context) -> bool`. The engine reasons in these three capabilities. The docstrings describe intent; the method name and behavior form the practical contract.

Inheritance: specializing the contract

`class HttpAction(Action)` declares that HttpAction is a kind of Action. It inherits the abstract base relationship and implements execute. `ManualTrigger(Trigger)` and `EqualsCondition(Condition)` do the same for their families. This is hierarchical inheritance: multiple concrete subclasses share a base.

There is also multiple inheritance in the persistence layer: `class User(TimestampMixin, Base)` combines timestamp columns with SQLAlchemy declarative behavior. Do not confuse inheriting a mixin for code reuse with the engine's behavioral substitutability.

Polymorphism: the important line

```
for action in workflow.actions:
    action.execute(context)
```

The spelling is the same for every action, but the executed method depends on the concrete object at runtime. That is dynamic dispatch and subtype polymorphism. The engine never asks whether an action is email or GitHub. Adding a new action requires implementation and factory wiring; it does not require a new branch in this loop.

Python also supports duck typing: an object with a compatible execute method can work at runtime even without inheriting Action. The failure test deliberately uses such an object. Type annotations communicate intended contracts, while runtime Python behavior remains dynamic.

Encapsulation: bundling state and behavior

HttpAction keeps URL, method, headers, and body together and exposes execute. GmailService hides encryption and refresh details behind send. A leading underscore, as in `_decrypt`, signals an internal implementation convention; Python does not enforce Java-style private access here. Public attributes mean the project demonstrates encapsulation by responsibility and API boundaries, not strict access modifiers.

Composition: a workflow has components

The runtime Workflow constructor accepts a name, one Trigger, an optional Condition, and a list of Actions. A workflow **has** these objects; it does not inherit from every possible action. It is configurable because the same Workflow class can hold different combinations.

In strict UML terminology, ownership and lifetime would need careful modeling. In Python the project passes ordinary references, so describing it as object composition is appropriate without claiming enforced exclusive ownership.

Composite pattern: conditions contain conditions

AndCondition and OrCondition both implement Condition and hold an iterable of child Conditions. A leaf compares a field. A group evaluates children. Because both share evaluate, a group may contain another group, forming a tree.

```
AND
  equals(action, "opened")
  OR
    equals(priority, "high")
    contains(labels, "bug")
```

This evaluates as `action == opened AND (priority == high OR labels contains bug)`. Recursion lives in ConditionFactory when nested configuration becomes objects, and in group evaluation when nested objects call children.

Factory pattern: constructing objects from saved data

The database stores `{"type": "equals", "config": {"field": "repo", "value": "FlowForge"}}`. ConditionFactory reads the discriminator and constructs EqualsCondition. A class itself can be a dictionary value, which is why the simple-condition map can look up `condition_class` and then call it.

These are simple static factory methods, not a fully developed GoF Factory Method inheritance hierarchy or Abstract Factory family. "Factory pattern" is a useful general description; use the more precise "simple factories for configuration-driven construction" if challenged.

Strategy and dependency injection

Trigger, condition, and action implementations act as interchangeable strategies. Constructor injection supplies the workflow's collaborators. GmailService is supplied to actions during hydration rather than created in WorkflowEngine. FastAPI's Depends is another form of dependency injection at request boundaries.

Do not claim the code implements every pattern mentioned in research notes. It is event-driven in the broad sense of responding to events, but it has no explicit Observer subject/subscriber registry or event bus.

SOLID, honestly applied

Principle	Evidence	Qualification
Single responsibility	Engine orchestrates; repository queries; service coordinates persistence	Some route and concrete-action responsibilities still cross layers
Open/closed	A new action leaves the execution loop unchanged	Factory registration and validation still change
Liskov substitution	Concrete actions are intended to satisfy execute's contract	Incorrect subclasses can still violate behavior; annotations do not prove correctness
Interface segregation	Three small interfaces instead of one giant automation interface	No separate implemented Integration ABC
Dependency inversion	Engine works with abstract behavioral objects	Concrete actions depend on configuration and GmailService infrastructure

Do not say OOP is the only possible solution. Functions and a dispatch dictionary could implement a smaller engine. Classes were chosen because distinct stateful components, reusable contracts, composition, and the academic objective align well here.

8. Events, context, triggers, and conditions in detail

Event has type, source, payload, and timestamp. Type describes the normalized kind, such as manual or webhook; source describes origin, such as manual or github; payload is the input data; timestamp records Event construction time. Normalization gives the engine a common envelope. It does **not** flatten every provider's JSON into an identical schema.

ExecutionContext contains the Event, optional workflow owner ID, and mutable data used during one execution. `lookup(field_name)` checks context.data first, then uses `event.payload.get(field_name)`. This lets computed execution data override input fields. It performs only a top-level lookup: the string `repository.full_name` is treated as one literal key, not traversed as a dotted path.

Trigger truth table

Trigger	Match rule	Important detail
ManualTrigger	event.type is manual OR event.source is manual	More permissive than checking type alone
WebhookTrigger	event.type must be webhook; optional configured source must match	Missing source restriction accepts any webhook source in core
ScheduledTrigger	event.type is scheduled	Does not itself wait, schedule, or generate an event

The API's run_manual method creates `Event(type="manual", source="manual", payload=payload)`. The dashboard's `{source: "dashboard"}` is **inside payload**. Event.source and payload["source"] are distinct fields; do not confuse them.

Conditions

Equals compares the looked-up value using Python equality. NotEquals uses inequality. GreaterThan and LessThan attempt comparison and return False on TypeError. Contains tests membership and returns False if the candidate cannot support that operation. Contains on a string tests substring membership; on a list it tests element membership; on a dictionary it tests keys. Matching is normally case-sensitive.

Examples: `3 > 2` is True, but comparing a string to an integer with > raises TypeError and becomes False here. Missing fields return None. `None != "closed"` is True, so a not-equal condition can pass when the field is missing. Python equality also has numeric edge cases, such as `True == 1`; the current schemas do not make every condition strictly typed.

The UI stores the expected condition value as a string. It exposes equals, contains, and not_equals, but not numeric comparisons or group editors. The backend accepts more. Editing a workflow created through the API with several conditions/actions loads only the first of each into the current form; saving can replace the richer configuration. This is a UI limitation, not an engine limitation.

9. The engine, statement by statement

`ExecutionStatus(StrEnum)` defines string-valued symbolic statuses. StrEnum requires a sufficiently recent Python version; Python 3.11 introduced it. EngineResult is a frozen dataclass containing status, count of completed actions, and optional error.

`execute(workflow, event, user_id=None)` first calls the workflow trigger. A mismatch immediately returns SKIPPED with "Trigger did not match". No action runs. Next it constructs an ExecutionContext carrying the event and workflow owner. If a condition exists and evaluates false, it returns SKIPPED with "Conditions did not match".

It then sets executed to zero, enters a try block, and iterates actions in order. Each action receives the same context. Only after an action returns normally does executed increment. If action 2 fails after action 1 succeeds, action_count is 1; later actions do not run. An exception becomes FAILED with `str(exc)` or a fallback message. If the loop finishes, the result is SUCCESS.

There are several subtleties. Trigger and condition evaluation occur **outside** the action try block. Exceptions there may escape rather than become a FAILED EngineResult. An empty action list produces SUCCESS with count zero because no iteration fails. The engine does not check enabled; WorkflowService does. Context action_results are not included in EngineResult, and the service persists only action_count, so provider metadata appended by actions is currently transient.

Returning `str(exc)` helps users see provider status failures, but it is not an automatic secret sanitizer. Concrete actions try to raise safe messages. A future action could leak sensitive details through its exception. Explain that as a review obligation and hardening opportunity.

Complexity

Ignoring database and network latency, running a workflow with C evaluated condition nodes and A actions costs approximately O(C + A). Short-circuiting may visit fewer conditions. Recursive nested conditions require stack space proportional to tree depth. A generic statement like "O(1) because there is one engine" is incorrect.

Atomicity versus external side effects

A database rollback cannot undo an email already sent or a GitHub dispatch already accepted. If an external action succeeds and database commit later fails, history may be absent even though the side effect happened. If an HTTP request times out after the provider performed the operation, a retry may duplicate it. This MVP provides sequential attempts and records ordinary outcomes; it does not provide exactly-once execution or distributed transactions.

10. Database models and the meaning of 'model'

In FlowForge, model can mean three different things. An **ORM model** maps Python attributes to database tables. A **Pydantic model/schema** validates the shape of API data. A **runtime domain object** represents executable behavior and configuration. None of these is a machine learning model.

Relational vocabulary

A table contains rows; a column stores one attribute; a primary key uniquely identifies a row; a foreign key refers to another table's key; a unique constraint prevents duplicate values. A relationship describes how ORM objects connect. A relationship attribute is not itself the same as the underlying database foreign-key column.

SQLAlchemy's `Mapped[str]` is an ORM-aware annotation. `mapped_column(String(36), primary_key=True, default=new_id)` maps a text column and supplies a callable default. Writing `default=new_id` calls the function for each new row when needed; `default=new_id()` would evaluate too early. UUIDs are stored as strings for SQLite/PostgreSQL portability, not as PostgreSQL-native UUID columns in this code.

Every table

Table / class	Purpose and important columns
users / User	Local identity reference: UUID-like string ID, unique email, created_at
workflows / Workflow	Owner user_id, name, enabled, created_at, updated_at
workflow_triggers / WorkflowTrigger	One trigger per workflow, trigger_type, JSON config; workflow_id is unique
workflow_conditions / WorkflowCondition	Condition type/config and workflow_id; nullable parent_id exists but hydration does not reconstruct a tree from it
workflow_actions / WorkflowAction	Ordered steps with position, action_type, JSON config; optional integration_id
integrations / Integration	Generic provider/config storage associated with a user; not the implemented Gmail token table
gmail_connections / GmailConnection	One per user; email, encrypted access and refresh tokens, expiry
gmail_oauth_states / GmailOAuthState	SHA-256 state hash, owning user, expiry, optional used_at
executions / Execution	Workflow ID, status, start/end timestamps, trigger_data JSON, result JSON, error

Relationships and cardinality

```
User 1 ---------------- many Workflow
Workflow 1 ------------ zero/one WorkflowTrigger
Workflow 1 ------------ many WorkflowCondition
Workflow 1 ------------ many WorkflowAction (ordered by position)
Workflow 1 ------------ many Execution
User 1 ---------------- many Integration
Integration 1 --------- many WorkflowAction (optional association)
User 1 ---------------- zero/one GmailConnection
User 1 ---------------- many GmailOAuthState rows (via foreign key)
```

`back_populates` connects the two ORM navigation directions. `uselist=False` makes the trigger relationship scalar; the unique foreign key also enforces at most one trigger row per workflow. The schema does not enforce that every workflow must have a trigger; the service checks that requirement.

`cascade="all, delete-orphan"` on workflow-owned children means ORM deletion/replacement can delete those child rows. This is an ORM cascade, not an explicit database `ON DELETE CASCADE` declaration. A previously fixed bug involved deleting a workflow that already had execution history; adding the executions cascade solved that path and a regression

test now covers it.

Why relational tables plus JSON?

Relationships such as ownership, ordering, and execution linkage benefit from relational structure. Provider-specific configuration varies, so JSON avoids a column for every possible action field. This is a hybrid design, not full normalization of every configuration property. The trade-off is weaker database-level validation and harder querying of provider-specific fields.

JSON is configuration, not executable Python. Factories intentionally recognize supported type names instead of loading arbitrary classes or calling eval. Pickling runtime objects would couple storage to Python implementations and can create code-execution risks when untrusted data is deserialized.

SQLite versus PostgreSQL

SQLite is a local file/in-memory database with little setup, useful for tests and early development. PostgreSQL is a separate database server with stronger production concurrency and persistence options. SQLAlchemy reduces differences, but it does not make the two engines behaviorally identical. SQLite tests do not prove PostgreSQL migrations, timezone behavior, JSON behavior, concurrency, or all constraint enforcement.

The application calls `Base.metadata.create_all()` at startup. It creates missing tables, not a controlled history of schema upgrades. Alembic is a possible future migration tool but is not currently installed or used. A database URL selecting the psycopg driver may use `postgresql+psycopg://...`; a bare PostgreSQL URL's default driver behavior must be checked rather than assuming any installed driver is selected automatically.

11. Sessions, transactions, schemas, services, and repositories

The database dependency

`create_engine` creates SQLAlchemy's connection-management entry point. It is not WorkflowEngine. `sessionmaker` builds sessions associated with it. A Session tracks ORM objects and a unit of database work; it is not a Supabase login session.

`get_session` creates a session, yields it to request handling, commits after normal completion, rolls back on an exception, and always closes it. `flush` sends pending SQL to the current transaction so IDs and constraints are available; `commit` makes the transaction durable. Flush is not a final commit. Setting autoflush=False makes explicit flush calls important.

For SQLite, check_same_thread=False permits use across threads; it does not make every SQLite access pattern safe or automatically force all connections to share the same in-memory database. Tests and connection pooling need separate consideration.

For session and transaction semantics, see the SQLAlchemy session documentation. The descriptions here apply those concepts to `db.py`.

Schemas are the API contract

ComponentConfig has a type string and dictionary config. WorkflowCreate requires user_id, name, and trigger; enabled defaults false and lists default empty. WorkflowUpdate expects a replacement-style set of configuration fields, including enabled. The API uses PUT, not a partial PATCH implementation.

Name length is constrained to 1-200 characters. This does not strip whitespace by itself. The component type is a string, not a literal enum, so factories must reject unsupported types. Config uses broadly typed values and action constructors supply further validation. Pydantic is useful, but these schemas do not express every possible valid action configuration as a discriminated union.

WorkflowResponse is a summary. WorkflowDetailResponse inherits summary fields and adds full trigger/condition/action configuration. ExecutionResponse exposes status, input payload, result, and error. `model_validate(item, from_attributes=True)` reads attributes from an ORM object rather than requiring a dictionary.

Pydantic's model documentation explains this boundary-validation role. Ordinary Python hints elsewhere in the program do not provide the same validation automatically.

Repository methods

get loads a workflow by primary key. add adds and flushes it. list_for_user filters by owner and orders by updated_at descending. executions filters by workflow ID and orders by started_at descending. Query filters become SQL expressions; they are not Python loops over every existing row.

There is no pagination. Large histories would grow response size. Ordering by updated_at may also not reflect every child-only edit, because the parent timestamp depends on an actual parent UPDATE and no explicit bump is implemented for every configuration-only change.

Create, update, load, and run

Create verifies the local User exists, inserts a parent Workflow, attaches a trigger, creates condition rows, enumerates actions into positions, flushes, and validates that factories can build runtime components. If validation raises through the real request dependency, the transaction rolls back.

Update finds the parent, changes its fields, updates or creates its trigger, replaces condition/action collections, flushes, and validates. delete removes the parent and flushes; ORM cascades handle its configured dependents.

load_runtime fetches persisted configuration, constructs the trigger, constructs no condition/one condition/an AND group according to row count, creates ordered actions, injects GmailService, and returns both the stored row and a runtime Workflow. The stored row supplies IDs and ownership; the runtime object supplies behavior.

run_manual wraps its input as a normalized manual Event and delegates to run_event. run_event hydrates the configuration, rejects disabled workflows, calls the engine with the stored owner's user ID, constructs Execution, adds it, flushes, and returns it.

The timestamps started_at and completed_at are both assigned **after** engine execution. They therefore do not measure actual execution duration accurately. The result saves only action_count; input payload and error are saved without a general redaction layer. Disabled runs raise a ValueError rather than creating a SKIPPED record.

12. Every HTTP endpoint and its behavior

Method and path	Purpose	Authentication/behavior
GET /health	Returns status and service name	Lightweight liveness; does not prove database/provider readiness
GET /api/workflows?user_id=...	Lists that user's summaries	Owner ID must match authenticated identity when configured
POST /api/workflows	Creates configuration	201; synchronizes a local user from verified identity if needed
GET /api/workflows/{id}	Full editable configuration	404 missing, 403 other owner when auth configured
PUT /api/workflows/{id}	Replaces configuration	Validates through service/factories; returns detail
DELETE /api/workflows/{id}	Deletes workflow and ORM-owned children	204 with no body
GET /api/workflows/{id}/executions	Lists stored history	Checks workflow existence and owner
POST /api/workflows/{id}/run	Requests manual execution	Receives a payload object; returns the execution
POST /api/webhooks/{provider}	Receives GitHub events	Only github supported; optional configured HMAC verification
GET /api/integrations/gmail/connect	Creates OAuth state and consent URL	Requires authenticated user even in otherwise local-mode application
GET /api/integrations/gmail/callback	Exchanges authorization code	Validates stored state; redirects browser with outcome
GET /api/integrations/gmail/status	Reports connection flag and email	User-specific, does not return tokens
DELETE /api/integrations/gmail	Removes stored connection	Requires identity; current handler returns normal default response, not an explicit 204

The planned standalone `/api/executions/{id}` and separate enable/disable endpoints are absent. Enabled is updated through the workflow PUT request. FastAPI normally also exposes generated OpenAPI documentation at `/docs` and a specification at `/openapi.json` unless configured otherwise; the app does not disable those defaults.

Dependency injection syntax

`session: Session = Depends(get_session)` tells FastAPI to call the dependency and supply its yielded session. `authenticated_user = Depends(current_user)` invokes identity resolution before handler logic. The dependencies can be overridden in tests. That is why tests can replace the database or user without editing route code. See FastAPI dependencies.

`Header()` identifies a request header. FastAPI maps the parameter spelling `x_hub_signature_256` to the customary hyphenated header name. An annotated optional header is not automatically a verified identity; current_user must inspect and validate it.

HTTP errors versus execution results

HTTP 200 means the API request was handled; the returned execution may still have status FAILED or SKIPPED. HTTP 401 means missing/invalid authentication or signature. HTTP 403 means identity exists but authorization fails. HTTP 404 means missing resource/provider; the run handler also currently maps service ValueErrors, including disabled workflows, to 404. HTTP 400 is used for many configuration errors. Invalid Pydantic request shapes usually produce FastAPI's validation response, commonly 422.

These codes describe different layers. A provider's 403 may become an engine FAILED execution while FlowForge's HTTP response remains 200. The frontend should inspect the execution status, not assume every 200 means the automation succeeded.

The API wrapper handles 204 separately because attempting JSON parsing on an empty response body would fail. The `_detail` helper returns full action config, including a stored GitHub token to the authorized editor. A password input masks display; it does not remove that token from the response or storage.

13. Trace one complete manual workflow

Use this deterministic example first. It needs no real email delivery or GitHub side effect:

```
{
  "user_id": "<authenticated-user-id>",
  "name": "Dashboard demonstration",
  "enabled": true,
  "trigger": {"type": "manual", "config": {}},
  "conditions": [
    {"type": "equals", "config": {"field": "source", "value": "dashboard"}}
  ],
  "actions": [{"type": "noop", "config": {}}]
}
```

1. The user types into controlled React inputs. State holds name, trigger, condition fields, and action fields. Nothing has been saved yet.
2. Submit invokes saveWorkflow. preventDefault prevents the browser's ordinary full-page form submission. The function validates inputs and assembles a plain JavaScript payload.
3. `api.createWorkflow` serializes the object and calls the shared request function. That function reads the Supabase session and includes its access token in Authorization when available.
4. FastAPI matches POST /api/workflows. Pydantic constructs WorkflowCreate. Dependencies supply a database session and resolve identity.
5. The route checks the submitted user_id against the authenticated ID. It creates the local User reference if the verified user does not yet exist locally.
6. WorkflowService creates rows for workflow, trigger, condition, and action. The action position is zero. Factory validation ensures the supported components can be constructed.
7. The route returns the summary. The session dependency commits on successful completion. The frontend refreshes the list and clears the builder.
8. The user selects the workflow. The UI fetches full detail and execution history. It loads configuration into form state.
9. Run now sends `{"payload":{"source":"dashboard"}}` to the run endpoint. The server's Event envelope has type manual and source manual; the payload contains source dashboard.
10. The service loads saved rows and hydrates ManualTrigger, EqualsCondition("source", "dashboard"), and NoOpAction. The enabled check passes.
11. The engine checks the trigger, constructs context, looks up payload.source, sees equality, executes NoOpAction, and returns SUCCESS with count one.
12. The service stores an Execution with input payload, count one, no error, and final status. It flushes, returns it, and the request transaction commits.
13. The frontend fetches executions again and renders SUCCESS. That refresh is request-driven; there is no websocket subscription or continuous history polling.

Change the expected value to `other` and run again: the condition fails, the action does not execute, and SKIPPED explains why. Disable the workflow and run: the service raises an error instead of executing. Run a manual request against a webhook-trigger workflow: it normally produces SKIPPED because the normalized manual event does not match the webhook trigger.

14. Generic HTTP actions, network requests, and SSRF

What HttpAction does

At construction, it reads a URL from config, checks that it is a string, validates a public HTTP/HTTPS destination, normalizes the method to uppercase, accepts only GET/POST/PUT/PATCH/DELETE, constructs string-valued headers, and keeps a JSON-compatible body.

At execution, it constructs an HTTPX Request, checks the serialized request body size against 64,000 bytes by default, sends with a configured timeout, and retries transport failures or server errors up to the configured bound. The default max_retries of two allows **three total attempts**. A 4xx is not retried by this loop. A 429 therefore is not currently retried, and Retry-After is not handled. There is no delay, exponential backoff, jitter, or shared client pool.

The implementation appends only type and status_code to context action_results. It does not keep the response body there. The service currently persists only count, so even that metadata is not exposed through saved execution results.

Why a generic HTTP action is powerful

Many services offer APIs. A configurable HTTP action can call some of them without writing a dedicated class for each provider. The cost is that users must understand URLs, methods, headers, and JSON. It also increases the security surface because the server is making a user-directed network request.

SSRF explained

Server-Side Request Forgery occurs when an attacker convinces a server to request an unintended destination. `127.0.0.1` is the server's own loopback interface, not the user's laptop. Private IPs may reach internal services, and link-local addresses can expose cloud metadata in some environments.

`validate_public_url` parses the scheme and hostname, resolves DNS with getaddrinfo, converts each returned address to an ipaddress object, and rejects private, loopback, link-local, reserved, or unspecified addresses. Redirect following is disabled so a public URL cannot simply redirect the HTTPX client to an internal address.

These are real guards, not a proof of complete SSRF resistance. DNS is resolved during validation and may be resolved again during connection; DNS rebinding remains a potential gap. There is no explicit egress firewall or destination allowlist, no separate response-size cap, and no explicit rejection of all multicast cases. The resolver uses port 443 as its default even for an HTTP URL; actual request semantics still come from the URL. Headers config is not comprehensively type-validated before `.items()`, so malformed shapes can raise unexpected errors.

Response redirects are not followed, but a 3xx is also not rejected by `response.is_error` because that flag covers client/server errors. Thus a redirect response can currently count as success even though no redirected action occurred. Explain the exact behavior instead of claiming redirects are followed or always failed.

See OWASP SSRF prevention guidance for defense concepts, and HTTPX timeout documentation for the distinction between network-operation timeouts and an overall workflow deadline. The configured timeout is not a guaranteed ten-second total cap across all attempts and actions.

15. GitHub: incoming webhook and outgoing dispatch

Incoming: GitHub calls FlowForge

An event occurs at GitHub. GitHub sends an HTTP POST with JSON bytes to `/api/webhooks/github`. The handler first rejects other provider names, reads raw bytes, and, if a backend secret is configured, computes an HMAC-SHA256 digest and compares the `X-Hub-Signature-256` value with compare_digest.

HMAC is a message authentication code using a shared secret, not encryption and not an asymmetric digital signature. SHA-256 alone hashes data; HMAC combines hashing with a secret to authenticate it. The expected header is `sha256=` plus a hexadecimal digest. `encode()` converts a secret string into bytes. Computing the digest over the original bytes matters because reformatting JSON changes the byte sequence even if the parsed object is equivalent. compare_digest reduces timing information leakage during comparison. GitHub's webhook verification documentation describes this scheme.

After verification, the route parses JSON, requires an object, loads all enabled workflows, filters for webhook triggers, and runs each with `Event(type="webhook", source="github", payload=payload)`. It returns a list of execution responses.

Important limits: verification is skipped if the configured secret is empty; there is no per-workflow secret or repository-to-owner routing; one valid incoming event is offered to all enabled webhook workflows; GitHub event headers are not normalized into a separate event-kind field; there is no delivery-ID deduplication, replay protection, request-body cap, or background queue. The returned execution list can expose candidate workflow payload/results to the webhook caller. This is a constrained MVP endpoint, not a finished multi-tenant webhook gateway.

Payload trap that often causes SKIPPED

GitHub normally sends repository as a nested object. A condition `repository == "owner/repo"` compares an object with a string and fails. A condition field `repository.full_name` also fails because lookup does not traverse dots. For a predictable simple demonstration, inspect the real payload and choose a top-level string such as action when appropriate, or use no condition. Do not tell the examiner the code extracts repository.full_name automatically.

Outgoing: FlowForge calls GitHub

GitHubAction reads a token, owner/repository, optional endpoint, event type, and body. The normal path sends POST to `/repos/{owner}/{repo}/dispatches` with `event_type` and `client_payload`, plus GitHub API headers and bearer authorization. Dispatch is an event notification to GitHub, not directly an issue-creation API call.

The included `.github/workflows/flowforge-dispatch.yml` listens for repository_dispatch type flowforge. A job runs on an Ubuntu runner and uses `gh issue create` to create a visible issue. That workflow's `github.token` is a GitHub Actions runtime token, distinct from the personal access token FlowForge used to send the dispatch. The example workflow grants issues:write for the issue step. Values from the event payload are passed through environment variables rather than interpolated as executable shell code directly into the run body.

A successful dispatch response means GitHub accepted the dispatch; it does not prove the downstream workflow completed successfully. Check GitHub Actions and the resulting issue separately. Token permissions depend on token type and endpoint; for the documented fine-grained repository-dispatch setup, consult GitHub's repository dispatch reference.

GitHubAction contains an additional issues/comments endpoint option, but the constructed path lacks a particular issue number and is not a supported complete create-comment flow. The visible UI supports repository dispatch. Do not present issue commenting as a verified feature.

Tokens in GitHub action config are stored as ordinary JSON and returned in workflow detail for editing. They are not encrypted by the Gmail Fernet logic. They are kept out of ordinary action result metadata, which is a much narrower protection. This distinction is a strong answer to a security follow-up.

16. Email: Resend and Gmail are different paths

EmailAction validates recipients, subject, and content. It accepts a recipient string or nonempty list, caps recipient count at the configured value (50 by default), uses a basic regex plus parseaddr, requires a nonblank subject up to 200 characters, and requires text or HTML. This is pragmatic input validation, not full international email standards validation or a guarantee that a mailbox exists.

The provider is selected by backend EMAIL_PROVIDER, not separately for each workflow. Resend requires a backend API key and sender. It builds a JSON request to the provider, caps serialized message size at 100,000 bytes by default, and uses bounded retry behavior similar to HttpAction. It saves only selected provider metadata in context. It does not save recipients, body, or API key as action results.

Gmail uses the workflow owner's stored OAuth connection. EmailAction requires a GmailService and context user_id during execution. It passes recipients and content to that service. The connected Gmail address determines the sender, regardless of the optional sender parameter. The constructor has an ineffective assignment of None to gmail_service when it is already None; the real missing-service rejection happens during execute.

The Gmail path does not apply the same message-size check or retry loop as the Resend path. General email validation is shared, but delivery guarantees and limits are not identical. No path guarantees receipt in the inbox or bypasses provider quotas/spam filtering.

Why OAuth instead of storing a Gmail password?

OAuth delegates permission. The user approves a limited capability and the application receives tokens rather than the user's Google password. A short-lived access token authorizes API requests; a refresh token can obtain new access tokens. Supabase login and Gmail connection solve different problems: the first identifies the FlowForge user; the second authorizes access to that user's Google account.

The complete Gmail authorization flow

1. An authenticated user clicks Connect Gmail. The frontend requests the backend connect endpoint.
2. The backend makes a cryptographically random URL-safe state value, saves its SHA-256 hash with the user ID and a ten-minute expiry, and returns a Google authorization URL.
3. The browser navigates to Google. The URL includes the client ID, exact callback URI, response_type=code, scopes, access_type=offline, prompt=consent, and state. Client ID identifies the app; it is not the client secret.
4. Google asks for consent. The scopes include openid, email, and gmail.send, not general inbox-reading permission.
5. Google redirects the browser to the backend callback with code and state, or an error.
6. The callback hashes the received state, finds the stored row, checks expiry and used_at, and marks it used. The stored user ID binds this connection to the FlowForge user who initiated it.
7. The server exchanges the code at Google's token endpoint using its client secret and redirect URI. It validates the returned access token, refresh token, and expiry.
8. It requests the account profile to learn the email address and encrypts tokens before saving the connection.
9. It redirects to the frontend with a status flag. The browser still relies on its own Supabase session to access user-specific endpoints.

The state value addresses authorization-response forgery/linking attacks. It is not the same as the access token and is not used to encrypt credentials. Google flow details are in the OAuth web-server documentation.

Encryption, hashing, and encoding

Fernet is authenticated symmetric encryption: the secret key enables encryption and decryption and detects tampering. The application needs to recover tokens to call Google, so encryption is appropriate. State needs comparison, not recovery, so hashing is appropriate. Base64 encoding merely represents bytes as text; it is not a secrecy mechanism. Fernet's documentation defines its security and key requirements.

Token encryption protects stored tokens if a database dump leaks without the key. It does not protect them from an attacker who has both the database and backend encryption key. Losing/changing the key without a migration prevents existing ciphertext from being decrypted. Multi-key rotation is a possible improvement; it is not implemented.

Sending with Gmail, byte by byte in concept

GmailService finds the owner's connection, decrypts tokens, normalizes naive expiry timestamps to UTC, and refreshes the access token when expiry is within one minute. It builds an EmailMessage with From, To, Subject, plain text, and optional HTML alternative. The standard email library produces MIME bytes; base64.urlsafe_b64encode converts them to Gmail's expected raw-message representation. The service POSTs `{"raw": encoded}` to users/me/messages/send and returns selected metadata. The `me` resource refers to the authorized token's account.

The Gmail sending guide describes MIME and base64url requirements. FlowForge uses Python's standard email library rather than manually constructing MIME boundaries.

Disconnect deletes the local encrypted connection. It does not call Google's token revocation endpoint or remove already granted consent in the Google account. Expired OAuth-state cleanup, PKCE, concurrent refresh coordination, and comprehensive refresh/replay tests are not implemented. Some OAuth exchange error detail is forwarded through redirect query parameters, so a general provider-error sanitization policy would improve privacy.

17. Authentication, authorization, and browser security

Sign-in in the frontend

The Supabase module creates a client only if its URL and anonymous/publishable key exist. Login calls signInWithPassword with entered email and password. The SDK handles the remote authentication flow and session state. The App subscribes to auth changes and updates React state. It also retrieves any initial session. The effect returns unsubscribe so the subscription is cleaned up.

The interface has sign-in and sign-out, but not self-service registration or password reset. Accounts must be provisioned separately for the MVP. Sign-in API behavior is described in Supabase's JavaScript reference.

Verification in the backend

The backend does not trust the supplied user_id alone. When configured, current_user requires a bearer header and calls Supabase `/auth/v1/user` with that token and the project API key. It checks the response and returns an AuthenticatedUser with ID and email. The code delegates token validation to Supabase; it does not locally decode and cryptographically verify JWTs using a JWKS cache.

Authentication asks 'who are you?' Authorization asks 'may you operate on this workflow?' Routes compare the verified ID with the requested owner or stored workflow owner. A hidden button in the UI would not substitute for that backend check. A guessed UUID should not grant access.

If supabase_url is empty, current_user returns None, allowing local unauthenticated workflow operations. Production must configure it correctly. This is a fail-open configuration mode, not an unconditional guarantee that every deployed request is authenticated. Gmail endpoints add their own requirement for a non-None user.

CORS

CORSMiddleware allows the configured frontend origin plus local development origins, credentials, methods, and headers. A browser may send an OPTIONS preflight before the actual request because of cross-origin JSON or Authorization headers. The server must allow the exact origin. A trailing slash or different deployment hostname can cause a mismatch.

CORS is enforced by browsers. It is not a substitute for authentication, does not stop server-to-server requests, and does not guarantee a valid webhook. HTTPS and CORS address different problems.

Frontend environment values are public

Vite replaces `import.meta.env.VITE_*` values during the build. The API URL and Supabase public project key are expected to be visible to the browser. Google client secrets, provider API keys, service-role keys, and Fernet keys must remain backend-only. Changing a Vercel frontend environment value generally requires a new build for it to enter the shipped bundle. See Vite environment variable documentation.

An anonymous/publishable key identifies a public client and relies on the service's access controls. It is not equivalent to a privileged service-role key. FlowForge's main workflow database access goes through SQLAlchemy and backend owner checks; do not claim that the source implements Supabase row-level-security policies or that RLS is what enforces these route checks.

React ordinarily escapes text interpolation, which helps avoid HTML injection in its own UI. The app does not use dangerouslySetInnerHTML to display workflow input. Email HTML is nevertheless sent as user-authored content to another system; React escaping does not sanitize an email body.

18. React, TypeScript, and JSX explained from the source

Entry point

index.html supplies `<div id="root"></div>` and loads main.tsx as a module. `document.getElementById('root')!` finds the DOM element; `!` is a TypeScript non-null assertion, not a runtime check. createRoot attaches React rendering to it. `<App />` renders the component. StrictMode enables extra development checks, including effect setup/cleanup checks; it does not render two visible production applications.

App is a function returning JSX, a JavaScript syntax extension for UI markup. React turns the returned structure into DOM updates. A state change requests another render; direct mutation of an ordinary local variable does not provide the same React state mechanism.

State syntax

```
const [selected, setSelected] = useState<WorkflowDetail | null>(null)
```

const declares a binding. Array destructuring extracts two returned entries. selected is the current value; setSelected requests an update. The generic describes the allowed state type. The argument null is the initial value, which is why code must guard against no selection. The setter does not immediately rewrite all values inside the currently executing function; React schedules the next render.

Most state variables correspond to form inputs. The form uses **controlled inputs**: value comes from state and onChange updates state. Checkbox inputs use checked and event.target.checked. A password input masks characters visually but does not encrypt the in-memory value.

Effects and cleanup

`useEffect(callback, [])` synchronizes after mounting and does not rerun because of ordinary dependency changes; development StrictMode may run an extra setup/cleanup cycle. The auth effect loads a session and subscribes to future changes. Another effect checks Gmail callback query parameters. Effects depending on session?.user.id refresh workflows and Gmail status after the signed-in identity changes.

An effect is not a replacement for every function. Saving happens in a form event handler because the user initiated it. The current lint warning concerns setting error state directly in the Gmail-query effect. That warning is an opportunity to reconsider state derivation, not evidence that the build failed.

Async and error handling in the UI

An async function returns a Promise. await pauses that function until a promise settles, allowing the browser to continue other work. try/catch/finally shows an error and clears busy even on failure. `void someAsyncFunction()` discards the returned value intentionally; it does not cancel the work, make it synchronous, or catch a rejection by itself.

`caught instanceof Error ? caught.message : 'fallback'` narrows an unknown thrown value before reading message. Errors need not always be Error objects in JavaScript. Some paths, such as initial getSession/signOut promises, do not have comprehensive catch handling.

Important TypeScript and JavaScript operators

Syntax	Meaning in FlowForge
import type	Imports types for checking; removed from runtime output
Record<string, unknown>	Object with string keys and values needing inspection before assumptions
Workflow & {...}	Intersection combining summary fields with detail fields
Component[]	Array of component configuration values
optional?: T	Argument/property may be absent
x?.y	Optional chaining; returns undefined if x is null/undefined
x ?? fallback	Uses fallback only for null/undefined, preserving false and empty string
x || fallback	Uses fallback for any false-like x; different from ??
condition ? a : b	Conditional expression choosing one value
{...obj}	Object spread; later properties can override earlier ones
(item) => ...	Arrow function, used for callbacks and transformations
value as T	Compile-time assertion; no runtime validation
typeof value === 'string'	Runtime check that supports type narrowing
item is string	User-defined type predicate, used when filtering recipients
===	Strict equality without ordinary loose-equality coercion
`text ${value}`	Template string interpolation
/\/$/	Regex matching a slash at the end of a string

The language syntax is explained in the TypeScript handbook. TypeScript assertions in the API wrapper do not guarantee the server actually returned the promised JSON shape.

Rendering syntax

JSX uses className instead of HTML class, braces for JavaScript expressions, and capitalized component names. `error && <p>...</p>` conditionally renders a message. `selected ? ... : ...` chooses an editing or creation label. `<>...</>` is a Fragment that groups siblings without adding a DOM wrapper.

`workflows.map(...)` turns data into elements. `key={workflow.id}` gives React stable sibling identity when lists change. An array index would be a weaker key when rows are reordered or deleted. A key is not an HTML ID and is not passed as an ordinary prop.

`onClick={() => void runWorkflow()}` passes a callback for a later click. Writing `onClick={runWorkflow()}` would invoke the function while rendering and pass its result instead. Buttons inside forms normally submit unless type="button" is specified; Cancel and Delete explicitly use button type to avoid accidentally saving.

App functions

refreshWorkflows updates the summary list. login signs in. loadBuilder loads selected detail or resets defaults. selectWorkflow fetches detail and history. saveWorkflow validates action-specific fields, parses JSON, constructs the payload, chooses create/update, refreshes, and resets. deleteWorkflow requests browser confirmation and deletes. runWorkflow requests a manual run and refreshes history. connectGmail gets an authorization URL and navigates away. disconnectGmail deletes the connection and updates local display.

`JSON.parse` input is treated as unknown and checked to be a non-null, non-array object. This avoids assuming a user entered the right shape. It is still only frontend validation; the server repeats necessary trust-boundary checks. `.split(',').map(trim).filter(Boolean)` turns comma-separated recipients into nonempty trimmed strings.

The App is intentionally compact but places many concerns in one component. Splitting auth, builder, list, history, and Gmail UI into components/custom hooks would improve maintainability as complexity grows. Redux is not automatically necessary for this size; local state avoids extra machinery. No React Router or TanStack Query is installed.

For React's foundational component/state/rendering concepts, see the official React quick start. The specific limitations described above come from FlowForge's App.tsx.

19. The frontend API wrapper and CSS

request<T>

The generic request function centralizes the base URL, token propagation, JSON content type, error handling, and response parsing. T describes the expected return type. It reads a Supabase session, obtains access_token if present, spreads supplied request options, builds headers, and calls fetch.

The order of spreads matters: caller options appear before the explicit headers object, then caller headers are spread last within it. A caller can override a default header. Current callers pass ordinary options; the function is not a full robust merge implementation for every HeadersInit form.

fetch rejects on many network failures but does not automatically throw for HTTP 400/500. The wrapper checks response.ok. It attempts to read a JSON detail string; if unavailable it throws a generic Request failed. It special-cases 204 and otherwise returns response.json with a type assertion. There is no AbortController timeout, retry, schema validation, or global cache in this wrapper.

encodeURIComponent protects the user_id query value from being interpreted as query syntax. JSON.stringify constructs the request body. Body data is still untrusted at the server even if it arrived from your own UI.

CSS that actually styles the product

main.tsx imports index.css. App.tsx does not import App.css. Therefore App.css is an unused starter stylesheet; the React/Vite logos and hero asset are also not evidence of product features. The line companion labels that file as inactive.

index.css sets global typography, box sizing, colors, forms, cards, buttons, layout, error text, and the webhook setup panel. `:root` targets the root element. * targets all elements. `.card` is a class selector; `.workflow-row.selected` requires both classes; `button:disabled` uses a pseudo-class. `@media (max-width: 720px)` changes the two-column grid to one column and adjusts spacing for narrow screens.

Grid arranges major columns; flexbox arranges controls and headings. `minmax(0, 1fr)` permits content to shrink instead of forcing an overflow from intrinsic minimum widths. `box-sizing: border-box` includes padding and borders in declared dimensions. `min-height: 100vh` fills at least the viewport height. `rem`/em relate to font sizes; px is a CSS pixel rather than a guaranteed physical hardware pixel. The project mostly uses px and fractional grid units.

The CSS names Inter in its font stack but does not download an Inter font in this source; system fallback may be used. A media query is evidence of responsive intent, not evidence that this session tested every mobile device. Input placeholders do not replace accessible labels; some compact condition fields could use clearer dedicated labels.

20. Libraries, tools, versions, and alternatives

Versions below are **manifest declarations**, not a claim that all developers have the exact same installed package versions. Backend requirements use lower bounds; frontend caret/tilde ranges allow some updates. The successful build in this session identified Vite 8.3.3. Version labels shown by current online documentation may differ from the repository.

Dependency/tool	Declared requirement	Role and why it fits	Plausible alternative and trade-off
Python	No explicit pinned interpreter; StrEnum needs 3.11+	Readable OOP core and test ecosystem	Java/C++ demonstrate OOP too, but add different web/deployment overhead
FastAPI	>=0.115.0	Typed request handling, dependency injection, generated API schema	Flask offers a smaller core but needs more explicit validation/documentation wiring; Django offers a larger integrated framework
Uvicorn standard	>=0.32.0	ASGI server running the FastAPI app	Other ASGI servers are possible; framework and server are separate responsibilities
SQLAlchemy	>=2.0.36	ORM models, relationships, sessions, portability	Raw SQL offers explicit query control; Django ORM ties the project more closely to Django
psycopg binary	>=3.2.0	PostgreSQL DBAPI driver	Other PostgreSQL drivers need matching SQLAlchemy URLs and sync/async choices
Pydantic	>=2.9.0	Runtime API data validation/serialization	Manual checks require more boilerplate; dataclasses alone do not validate incoming JSON equivalently
pydantic-settings	>=2.6.0	Typed settings from environment and dotenv path	os.environ is simpler but needs manual conversion/default handling
HTTPX	>=0.27.0	HTTP clients, timeouts, mockable network boundary	requests is familiar and synchronous; urllib is standard-library but lower-level
cryptography	>=43.0.0	Fernet encryption for Gmail token storage	Managed secret storage improves operational control but adds setup; base64 is not an encryption alternative
pytest	>=8.3.0	Tests, fixtures, assertions, monkeypatching	unittest is built-in but usually more verbose for these fixtures
python-dotenv	>=1.0.1	Supports local dotenv loading used through settings	Host environment variables suffice in production; dotenv is not a secret manager
React	^19.2.8	State-driven interactive forms/dashboard	Vanilla JS is viable but needs manual state/DOM synchronization; Vue/Svelte offer different component approaches
react-dom	^19.2.8	Browser DOM renderer for React	React itself defines components; DOM rendering requires the web renderer
TypeScript	~6.0.2	Catches many mismatched data/prop usages while developing	JavaScript avoids a compile step but loses static checking
Vite	^8.3.0	Development server and production asset build	Next.js adds routing/SSR/server features beyond this SPA's current needs
@vitejs/plugin-react	^6.1.1	React-aware Vite transformation/development support	Other React transform integrations exist; the chosen plugin matches the scaffold

Dependency/tool	Declared requirement	Role and why it fits	Plausible alternative and trade-off
@supabase/supabase-js	^2.117.2	Browser auth client and session events	Custom auth requires secure password/session implementation; another managed identity provider changes setup/cost assumptions
@types/react	^19.2.18	Type declarations for React	Not runtime React code; needed for the TypeScript development experience
@types/react-dom	^19.2.7	Types for the DOM renderer	Same distinction: declarations versus browser behavior
@types/node	^24.13.3	Node-side tool/config type declarations	Does not make Node APIs available in a browser
Oxlint	^1.81.0	Static lint checks with React/TS rules	ESLint has a broad plugin ecosystem; actual lint script here runs Oxlint

These alternatives are engineering comparisons. The design documents explicitly discuss React/FastAPI versus Next.js and Django, and queue complexity. They do not record a historical team vote on every library above. Do not present inferred rationale as a documented personal decision.

Standard library modules also count as dependencies of understanding

abc supplies abstract classes; dataclasses generates data-object methods; enum supplies StrEnum; datetime/timezone/timedelta handles timestamps; uuid generates IDs; pathlib resolves file paths; typing and collections.abc express contracts; socket resolves hosts; ipaddress classifies addresses; urllib.parse parses URLs and constructs queries; re validates basic email shape; email builds/addresses MIME messages; base64 encodes message bytes; hashlib hashes state; hmac authenticates webhooks; secrets creates secure state randomness; unittest.mock replaces network dependencies in tests; contextlib supports lifespan cleanup.

Built-in syntax/functions include isinstance, len, str, list, dict, all, any, enumerate, range, bool, and exception classes. They are not separately installed packages. Native browser fetch, URLSearchParams, window.history, and document are web APIs, not imported npm dependencies.

Important indirect libraries and runtime boundaries

FastAPI builds on Starlette's ASGI web primitives and Pydantic's validation. ASGI is the server/application interface: Uvicorn receives network traffic and calls the application using that interface. Starlette supplies machinery such as routing, middleware, responses, and TestClient underneath the APIs used here. AnyIO supports asynchronous concurrency/bridging used by the test client. HTTPX uses HTTP transport machinery including httpcore. Pydantic uses pydantic-core for validation. These are supporting dependencies, not separate workflow engines you wrote.

The local verification environment reported Python 3.13.7, FastAPI 0.142.2, Starlette 1.7.0, AnyIO 4.15.1, SQLAlchemy 2.1.3, Pydantic 2.13.5, HTTPX 0.28.1, and pytest 9.1.1. This illustrates why installed versions differ from minimum requirements. It is an environment snapshot, not a requirement to upgrade another machine to those versions.

A browser's JavaScript event loop, Python's asyncio event loop, and a database transaction are different mechanisms. Async I/O allows other work while an operation waits; it is not automatically CPU parallelism. FastAPI can run ordinary synchronous handlers through thread machinery, but synchronous work called directly inside an async handler can block that handler's event loop. Adding the word async without changing blocking operations does not solve the problem.

Budget and hosting trade-off

The project targets a free-tier deployment: static frontend on Vercel, Python API on Render, database/auth on Supabase. Those are project targets, not a guarantee of today's provider pricing or quotas. A separate backend makes the Python engine visible and testable but adds CORS, environment, and deployment coordination. A single full-stack framework could reduce deployment boundaries but would change the chosen Python-centered design. Free/inactive services and external quotas constrain reliable scheduling and response latency.

21. Tests: what they prove and what they do not

The current backend suite contains 28 tests across 10 test modules, plus conftest.py fixtures. In this preparation session, all 28 passed with one warning when run outside the sandbox using a forced in-memory database. The first sandbox run stalled while Windows asyncio was creating its internal socket pair, before the Gmail callback test body; that was not evidence of a Gmail API failure. No application code or tests were changed to obtain the passing result.

Test module	Count	Main evidence
test_auth.py	1	Mocked Supabase identity response becomes AuthenticatedUser
test_config.py	1	Settings dotenv path is based on the backend file location
test_engine.py	5	Factory/group construction, comparisons, skip behavior, multiple actions, safe example failure
test_gmail.py	3	Mocked Gmail send, missing connection rejection, OAuth connect/callback persistence with mocked exchange/profile
test_health.py	1	Health endpoint's successful response
test_integrations.py	6	Private URL/scheme rejection, mocked HTTP/GitHub/email requests, recipient validation
test_models.py	1	Workflow/child configuration storage and retrieval
test_oop.py	4	Manual trigger matching/rejection, equality outcomes, action composition
test_workflow_api.py	5	Create/get/run, webhook dispatch, signature requirement, update/delete, deletion with history
test_workflow_service.py	1	Create/hydrate/run/persist flow through the service

A **unit test** isolates a small behavior. A **service integration test** connects service logic with a test database. An **API test** exercises routing and dependency wiring through TestClient. A **live end-to-end test** uses the real browser, identity configuration, database, and intended external providers. The suite includes the first three forms, with external provider boundaries mocked.

Fixtures, monkeypatch, and mocks

pytest discovers test_ functions. A fixture provides reusable setup/cleanup. db_session creates in-memory SQLite tables, yields a session, then closes and drops test tables. client runs the FastAPI app through TestClient. The autouse fixture disables configured Supabase auth by default and selects Resend so ordinary tests are deterministic.

monkeypatch temporarily changes settings or methods and restores them after a test. patch replaces a name where the code under test looks it up. MagicMock supports magic methods such as __enter__, useful for `with httpx.Client(...)`.

A response Mock supplies fields such as status_code and a programmed json result. No real email is sent merely because a mocked send test passes.

API tests use app.dependency_overrides to supply the test session or user. A finally block clears overrides so one test does not contaminate the next. `assert` checks an expected property and fails the test if false. The `else: raise AssertionError` pattern in a missing-connection test ensures an expected exception actually occurred.

Do not overstate coverage

The test named executes_actions_in_order checks both actions were called once and count is two; it does not record an interleaving trace proving exact order independently. The implementation's loop and ordered relationship provide that evidence. Similarly, `isinstance(x, object)` is a weak assertion because almost every Python value is an object.

The suite does not comprehensively test cross-user denial for every route, configured missing/bad auth cases, DNS rebinding, response limits, retry duplication, OAuth-state replay/expiry, token-refresh concurrency, PostgreSQL behavior, or browser interactions. There is no checked-in frontend unit-test suite or browser automation suite. Name these as improvement targets rather than pretending a test count proves universal correctness.

Checks performed for this report

- Backend: 28 passed, one warning; isolated SQLite database, external service behavior mocked by existing tests.
- Frontend: npm run build passed; its script executes `tsc -b` before the Vite production build, so both configured TypeScript projects were checked.
- Frontend lint: npm run lint completed successfully with one React set-state-in-effect warning at App.tsx line 51.
- Live browser/production services: not exercised in this documentation task; no claim of current deployment health or real email delivery.

Do not suppress a warning just to say "all clean." Discuss what it means and whether it is blocking. The session's focus was understanding/documentation, not changing product behavior.

22. Run commands and explain every part

Commands below are existing project operations. Run frontend commands in frontend and backend commands in backend with the intended virtual environment. The repository's RTK instruction prefixes shell commands; `rtk proxy` forwards a command without token filtering.

```
# From the repository root: force an isolated database in the test process
rtk proxy .\.venv\Scripts\python.exe -c "import os,sys,pytest;
os.environ['DATABASE_URL']='sqlite:///:memory:'; os.environ['SUPABASE_URL']='';
os.environ['GITHUB_WEBHOOK_SECRET']=''; sys.exit(pytest.main(['backend/tests','-q']))"

# From backend: API development server
rtk proxy ..\.venv\Scripts\python.exe -m uvicorn app.main:app --reload

# From frontend: UI development and checks
rtk npm run dev
rtk npm run lint
rtk npm run build
rtk npm run preview
```

python selects the interpreter. `-m pytest` asks it to run an installed module, keeping interpreter/package selection aligned. `-q` means quieter test output. `app.main:app` means import the Python module app.main and use its variable app as the ASGI application. `--reload` restarts the development server when watched source changes; it is a development feature, not a requirement for production.

npm reads package.json scripts. dev runs vite, build runs `tsc -b && vite build`, lint runs oxlint, and preview serves a built frontend for local inspection. `&&` in the npm script means the build step follows only if type checking succeeds. Do not invent npm test or npm typecheck scripts; neither exists. A root `tsc --noEmit` command alone can miss referenced projects when the root config contains no source files; the actual build's `tsc -b` follows project references.

The virtual environment isolates Python dependencies from unrelated projects. requirements.txt describes install requirements; package.json describes frontend packages and scripts. A lockfile, where present, records a more exact dependency resolution. Broad version requirements without pinning can produce environment drift.

Configuration files

vite.config.ts enables the React plugin. tsconfig.json references application and tooling configs. The app config targets modern JavaScript, includes DOM types, preserves module syntax, permits TS extension imports, sets noEmit, and uses the React JSX transform. The node config checks Vite configuration with Node types. noUnusedLocals/noUnusedParameters catch unused declarations; noFallthroughCasesInSwitch catches accidental switch fallthrough.

The current configs do not explicitly enable `strict: true`. Some type checking is active, but do not describe this repository as maximally strict TypeScript just because it uses TypeScript. skipLibCheck skips checking declaration files in dependencies, not all project code. `erasableSyntaxOnly` favors TypeScript syntax that can be removed without runtime transformation of language features.

.oxlintrc.json enables React, TypeScript, and Oxc plugins and hooks-related rules. backend/pytest.ini sets Python import/test locations. frontend/index.html remains a basic Vite shell with title frontend; a polished metadata/title pass would be a small product improvement.

23. Known limitations and how to defend them intelligently

An examiner may ask "What is wrong with your project?" A good answer identifies a concrete limitation, its consequence, and a proportionate improvement. Do not answer that the project is flawless or dismiss security problems as merely academic.

Limitation in source	Consequence	Plausible improvement, not implemented
Synchronous in-request execution	Slow providers hold requests; async webhook handler can block event loop	Separate bounded execution from ingestion; add durable jobs only when scope warrants
No automatic scheduler	ScheduledTrigger cannot make a job run at a time	Add a reliable external scheduler or an always-on scheduler with persistence
No deduplication/idempotency keys	Webhook redelivery or retry can duplicate side effects	Store delivery identifiers and action idempotency keys
Shared webhook candidate selection	Valid events can reach multiple users' workflows	Bind per-tenant endpoints/credentials to intended workflows/repositories
Optional signature/auth config	Misconfiguration permits unverified requests	Production startup validation and explicit safe modes
GitHub tokens in config	Tokens stored and returned to editor	Separate encrypted credentials and return only redacted/reference data
Raw payload and exception persistence	Sensitive input or unsafe future errors may be retained	Redaction policy, safe error types, retention controls
Validation and connection DNS differ	DNS-rebinding SSRF risk	Validated connection binding and network egress controls
No response-size cap or workflow deadline	Large/slow responses consume resources	Streaming limits and total execution budget
No per-action persisted results	Harder to inspect which action/provider failed	Structured execution-step records with safe metadata
Incorrect start/end measurement	Cannot calculate real duration from current fields	Record start before engine call and end after it
create_all instead of migrations	Schema evolution is uncontrolled	Versioned migrations with rollback/backup planning
One visible condition/action	UI cannot express all backend features	Explicit multiple-step/group editor and safe round-trip handling
No nested field traversal	Common GitHub conditions unexpectedly skip	Document typed paths and implement safe nested lookup
No pagination/rate limiting	Large histories or excessive requests hurt resources	Cursor/page limits and per-user/provider controls
Gmail disconnect only deletes locally	Google consent may remain granted	Provider revocation flow plus local deletion
Basic tests around auth/refresh	Some security regressions could escape	Negative authorization and OAuth lifecycle tests

A reasonable improvement order is: make production configuration fail safely; enforce webhook tenant routing and credential safety; add negative security tests; correct history/timing; improve the builder; add durable execution/scheduling only when the deployment can support it. This is a suggested prioritization, not work performed by this report.

24. A practical viva demonstration script

Before the room

Use an existing provisioned test account. Confirm the frontend and backend target the same Supabase project, the backend origin matches the actual UI origin, and the workflow uses the intended database. Start with a deterministic no-op workflow so internet/provider behavior does not obscure the engine explanation. Keep tokens masked and avoid opening secret-bearing environment files on a projector.

Six-minute walkthrough

Minute 1: State the problem and the Trigger -> Condition -> Action model. Explain why reusable configuration is preferable when the same execution structure serves multiple automations.

Minute 2: Sign in and create an enabled manual workflow with a no-op action. Use source equals dashboard if demonstrating a matching condition. Explain that form state is temporary until the API call saves it.

Minute 3: Select it, run it, show SUCCESS, and explain that history came from the backend database. Change the condition to a nonmatching value, save, reselect, run, and show SKIPPED with the reason. This proves rule gating, not only happy-path execution.

Minute 4: Open the three ABC contracts, then the engine's action loop. Explain abstraction and dynamic dispatch. Open factories and show how a stored type string selects a concrete class.

Minute 5: Show the ORM/runtime Workflow distinction and one execution record. Explain JSON config, hydration, and why runtime objects are not pickled. If configured and authorized for the demonstration, show an already prepared GitHub dispatch or Gmail run and verify its separate provider outcome.

Minute 6: State the test evidence and limitations: no automatic scheduler, synchronous execution, frontend subset, and credential/webhook hardening work. Close by explaining one extension path without changing the engine.

If the live demonstration fails

If login reports invalid API key, check that frontend Supabase URL and public key belong to the same project and rebuild after changes. If the browser reports Failed to fetch, inspect browser network status, API reachability, and CORS origin rather than assuming a Python syntax error. If a webhook run is SKIPPED, inspect its stored error, trigger type/source, payload types, and top-level lookup. If an action fails with 403, distinguish FlowForge authorization from provider permissions. If Gmail fails, check connection, OAuth callback matching, scopes, token expiry/key continuity, and provider status using safe diagnostics.

Historical issues in MEMORY are useful examples of debugging reasoning, not a guarantee those exact faults still exist. Never display or paste secret values to prove configuration.

25. Viva question bank: concise answers with follow-up depth

Product and architecture

1. What problem does FlowForge solve? Repeated cross-tool operations can be represented as reusable workflows rather than separate scripts. The project adds configuration, ownership, execution, and history around a common pipeline.

2. What is a workflow? A configured trigger, optional condition, and ordered action collection. The persisted model stores its definition; the runtime object holds executable components.

3. Trigger versus condition? A trigger checks whether this kind/source of event is relevant. A condition checks properties of the event/context to decide whether relevant events should proceed.

4. Action versus integration? An action expresses an operation; integration code communicates with a provider. The design separates the concepts, but current HTTP/GitHub actions contain provider communication, and GmailService is the more explicit service boundary.

5. Why must the engine run on the backend? Trusted credentials, owner checks, provider access, and execution history belong on the server. Browser state is user-controlled and may disappear or be modified.

6. Is this MVC? It has separation resembling presentation/application/data layers, but it is more precisely described as a layered SPA plus API with an OOP engine. React components are not server-rendered MVC templates here.

7. Is it microservices? No. There are separate frontend/backend deployments and external services, but the backend modules are one application, not independently deployed microservices.

8. Is it event-driven? It responds to normalized events, especially webhooks. It has no durable event bus, subscriber registry, or distributed stream platform.

9. Where is AI used? No model calls are in the product runtime. Development tools may have assisted implementation; that is different from a user-facing AI feature.

10. What is your strongest design decision? Persist configuration and hydrate objects through factories, keeping provider-specific branches out of the engine loop. Then demonstrate that exact path in code.

OOP and language concepts

11. Show abstraction. Trigger.should_execute, Condition.evaluate, and Action.execute define small behavioral contracts. Engine code uses the capabilities without provider internals.

12. Show inheritance. HttpAction inherits Action; ManualTrigger inherits Trigger; AndCondition inherits Condition. ORM classes also inherit a timestamp mixin and Base.

13. Show polymorphism. action.execute(context) dispatches to the method of the actual concrete object. The loop is identical for HTTP, email, GitHub, and no-op actions.

14. Show encapsulation. HttpAction bundles request configuration and behavior; GmailService hides token encryption/refresh behind send. Underscores express internal conventions, not enforced private access.

15. Why composition? A workflow combines independently variable components. Inheriting a separate workflow class for every trigger/action combination would multiply classes unnecessarily.

16. What is the Composite pattern here? AND/OR are Conditions that hold other Conditions, so leaves and groups share evaluate. Trees can nest recursively.

17. What kind of factory is implemented? Static simple factories mapping stored type/config to concrete objects. It is not a claim of an Abstract Factory hierarchy.

18. What does staticmethod do? It allows a class-associated operation without an instance self argument. Factory creation needs input data, not persistent factory-object state.

19. Is Python method overloading used? Not Java-style multiple methods selected by parameter signature. The project mainly uses overriding and runtime dynamic dispatch.

20. Can Action be instantiated? Not normally while execute remains abstract. A concrete subclass must implement the abstract method.

21. What is duck typing? Code can accept an object that supports the required method even without explicit inheritance. One failure test supplies a compatible execute method this way.

22. What is self? The instance on which a bound method operates. It lets each object retain its own fields/configuration.

23. Is a type hint runtime validation? Not generally. Pydantic, explicit checks, and actual operations provide runtime validation; annotations mainly document and enable tooling.

24. What is a dataclass? A class with generated methods for data fields. Event/context/result use it; persistence still uses SQLAlchemy models.

25. Why default_factory? To allocate a fresh mutable dictionary/list for each object rather than accidentally sharing one default object.

26. Why frozen EngineResult? Results should not be casually reassigned after execution. It provides shallow assignment protection, not deep immutability.

27. What is recursion here? ConditionFactory calls itself for child configurations, and composite conditions may call nested composites' evaluate methods.

28. Does the engine satisfy open/closed? New behavioral types can leave the loop unchanged, but factories and supporting UI/validation still need extension. It is localized change, not zero change everywhere.

29. Why not functions instead of classes? Functions are viable. Classes make independently configured components and shared contracts explicit, fitting extensibility and the course's OOP objective.

30. Is there garbage collection? Python manages object lifetime automatically; SQLAlchemy sessions and HTTP clients still require explicit close/cleanup for external resources. Garbage collection is not a transaction policy.

Data and execution

31. Why store JSON config? It is portable data that can be edited, inspected, and rehydrated. It avoids coupling persistent definitions to live Python objects.

32. Why not pickle? Pickle couples storage to implementation and can execute code during unsafe deserialization. The application only needs recognized configuration, not arbitrary object restoration.

33. Two Workflow classes? ORM Workflow represents stored rows/relationships; RuntimeWorkflow represents a trigger, optional condition, and actions ready for the engine.

34. What is an ORM? A tool mapping objects/attributes and relationships to relational tables and SQL operations. It reduces repetitive persistence code but does not eliminate SQL concepts.

35. Primary key versus foreign key? A primary key identifies a row; a foreign key references a row elsewhere. workflow.user_id connects ownership to users.id.

36. Why UUID strings? Easy portable identifiers across SQLite and PostgreSQL. UUIDs do not replace authorization and are not proof that guessing/access attacks are impossible.

37. Flush versus commit? Flush synchronizes pending SQL within the current transaction; commit completes the transaction. A flushed row can still be rolled back.

38. What does rollback undo? Database changes in the transaction. It does not undo email delivery or external API actions already accepted.

39. Why order action rows? Workflows are sequences. position plus the ORM relationship order_by preserves the configured execution order.

40. Multiple top-level conditions? WorkflowService combines them as AND. Explicit nested OR/AND is possible in condition configuration but not fully exposed by the builder.

41. Missing field behavior? lookup returns None. Equality usually fails against a normal expected value, but not_equals may pass. Numeric/member operations catch TypeError in their concrete classes.

42. Dotted field lookup? Not implemented. repository.full_name is a literal key in the current lookup.

43. Why SKIPPED instead of FAILED? A rule mismatch is an expected decision not to run. FAILED means an attempted action raised an exception.

44. What if an action fails? Later actions stop; completed action count is preserved; the engine returns FAILED with an error string. Previous external effects remain.

45. Zero actions? Currently allowed, producing SUCCESS and zero count when trigger/condition pass. Requiring at least one action would need schema/service validation.

46. Does disabled mean SKIPPED? Not in the current service. Disabled run attempts raise ValueError; webhook candidate selection excludes disabled workflows.

47. Are runs queued? No. The service calls the engine synchronously within the request.

48. What happens after a server crash? There is no durable job queue/recovery protocol. An in-flight run can be lost or have an external effect without a committed history row.

49. Exactly once? Not guaranteed. Duplicate deliveries, uncertain network outcomes, and retries need idempotency/deduplication mechanisms.

50. Are execution timings accurate? Both timestamps are assigned after engine execution, so they do not measure actual duration properly yet.

API, auth, and security

51. REST versus HTTP? HTTP is the protocol. REST is an architectural style using resources and standard operations; this API is resource-oriented HTTP/JSON with additional run/connect operations.

52. POST versus PUT? POST creates or invokes a processing operation here. PUT replaces workflow configuration at a known ID. There is no partial PATCH implementation.

53. HTTP 200 but FAILED? The API handled the run request successfully; the automation outcome is a separate status in its body.

54. Authentication versus authorization? Supabase establishes who the user is; backend owner comparisons decide what that user may access.

55. Why not trust user_id? Any client can modify a JSON/query parameter. The verified token identity must match the requested owner.

56. Are JWTs checked locally? No. The backend calls Supabase's user endpoint to validate the bearer token and obtain identity.

57. What if Supabase URL is missing? Workflow authentication is disabled for local mode. This is a deployment-sensitive configuration risk.

58. Is CORS authentication? No. CORS controls browser cross-origin access; server clients can still call an endpoint and must be authenticated independently.

59. What is preflight? A browser's preliminary OPTIONS request checking whether a cross-origin method/headers are allowed.

60. What is SSRF? Misuse of a server's network position through a user-controlled destination. FlowForge rejects several nonpublic address classes and disables redirects, but DNS-rebinding and resource-limit gaps remain.

61. Are secrets encrypted? Gmail tokens use Fernet. GitHub action tokens are ordinary JSON config. Do not generalize one path's protection to all credentials.

62. Does a password input protect the stored token? It masks display only. Storage, network exposure, authorization, and encryption require separate controls.

63. HMAC versus encryption? HMAC authenticates message integrity with a shared secret; encryption hides content. Webhook verification does not encrypt the payload.

64. Why hash OAuth state? The callback only needs to compare state with the stored value; it does not need to recover the original random string from storage.

65. Why encrypt refresh tokens instead of hashing them? They must be recovered to request new access tokens. Hashing is intentionally one-way and would prevent that use.

66. Is base64 encryption? No. It is reversible representation without a secret key.

67. Why compare_digest? To reduce timing information leakage compared with ordinary early-exit string comparisons.

68. Is the webhook always verified? Only if a secret is configured. It should be required for an exposed deployment.

69. Do signatures stop replay? Not by themselves. A valid signed payload can be delivered again; delivery-ID deduplication is not implemented.

70. Is input ever logged unsafely? Raw event payload is stored in execution history and engine errors use exception strings. There is no general redaction policy, so input/extension design must be reviewed.

Integrations and frontend

71. Webhook versus API request? A webhook is an HTTP API request initiated by another service when an event occurs. Incoming GitHub webhooks and outgoing GitHub dispatches travel in opposite directions.

72. Does GitHubAction create an issue directly? The supported demo sends repository_dispatch; the separate GitHub Actions workflow creates the issue.

73. Why does dispatch success not prove issue creation? Acceptance of the event and completion of the downstream job are separate stages with separate permissions/errors.

74. Why not call Gmail with a password? OAuth grants scoped delegated access with revocable tokens and avoids storing the user's Google password.

75. Access versus refresh token? Access tokens authorize API calls for a limited lifetime; refresh tokens obtain replacement access tokens. Both are encrypted at rest here.

76. Why offline access and consent? The app requests a refresh-capable authorization for later server-side sends. Google's response is still validated rather than assuming required tokens always exist.

77. What does Gmail disconnect do? Deletes the local stored connection. It does not revoke the provider-side grant.

78. Why React? The form/dashboard needs consistent rendering from changing state. React reduces manual DOM synchronization; it is not mandatory for a workflow engine.

79. Why TypeScript? It catches many shape/type mistakes before runtime. It cannot prove a fetched server response is valid just because it was asserted as T.

80. What does useEffect do here? Synchronizes session subscription, callback query handling, and user-dependent fetching with component lifecycle/state.

81. Why cleanup? Unsubscribe prevents stale subscriptions/duplicate callbacks after lifecycle changes. Resource management matters even with garbage collection.

82. Why key in map? Stable IDs help React preserve the correct identity of sibling elements when lists update.

83. What does optional chaining prevent? Access errors when a reference is null/undefined. It is not a general validation of every nested data shape.

84. Unknown versus any? unknown requires checks before arbitrary property/operation assumptions; any largely opts out. The frontend prefers unknown for parsed/config values.

85. Tailwind, Axios, Redux? Not installed in the current product. Actual code uses plain CSS, fetch, and local React state.

86. Why one component? It kept the MVP compact. Growth would justify smaller components/hooks; component extraction alone does not require a global state library.

87. Does history update live? It reloads on selection/manual execution. There is no real-time subscription for incoming webhook outcomes.

88. What is Vite versus React? Vite develops/builds assets; React renders and manages UI components. Neither is the Python server.

89. Why are frontend env values not secrets? Vite embeds exposed variables into browser assets. Only public configuration belongs there.

90. What would you add next? Choose a specific verified weakness, such as tenant-aware webhook routing or negative authorization tests, and explain its consequence and acceptance criteria before proposing more features.

Testing, deployment, and reflection

91. What does 28 passing tests mean? The existing automated assertions passed in the tested environment. It does not establish complete security, production deployment health, or real provider delivery.

92. Why mock external services? Tests stay deterministic, fast, credential-independent, and free of real side effects. Separate live integration checks are still needed.

93. Why SQLite in tests? Low setup and isolation. PostgreSQL-specific behavior requires additional testing against PostgreSQL.

94. Why create_all is not migrations? It creates missing structures but does not provide reviewed, versioned schema upgrade/rollback steps.

95. How would you add Slack? Implement an Action subclass, validate its configuration/credential boundary, register it in ActionFactory, test it with mocked HTTP, and expose safe UI fields if desired. Leave WorkflowEngine's loop unchanged.

96. How would you add nested JSON lookup? Define a path syntax and missing/type behavior, implement traversal in ExecutionContext.lookup, test nested dicts/lists and ambiguous literal keys, then update UI/help. Do not simply split all dots without defining semantics.

97. How would you scale? First measure candidate selection, request latency, provider limits, and database queries. Then add pagination, indexes, bounded concurrency, tenant routing, and durable jobs as justified. Kafka is not a default answer to every scaling question.

98. Describe a real bug and lesson. Deletion with history needed the executions ORM cascade and a regression test. Or explain bearer-token propagation and local user synchronization. Attribute these to repository history; do not claim unverified personal debugging work.

99. Did you use AI to develop? Answer honestly about your actual involvement. Explain that you can trace generated code, test assumptions, identify limitations, and reason about changes. Do not claim authorship or understanding you do not have.

100. Can you handle any possible question? No fixed guide guarantees that. The best method is to trace inputs, state, outputs, failures, and trade-offs, then say what you would inspect or test when unsure.

26. Rapid revision: the facts to recall before entering

Pipeline: configuration -> factories -> runtime objects -> engine -> execution row. **Layers:** React -> route -> service -> repository/models; engine is a behavioral collaborator of the service. **Core contracts:** should_execute, evaluate, execute. **Statuses:** SUCCESS/FAILED/SKIPPED. **Dispatch line:** action.execute(context).

Persistence: relational ownership/relationships plus JSON config. **Two models:** ORM Workflow versus RuntimeWorkflow. **Transactions:** flush is not commit; rollback cannot unsend mail. **Conditions:** top-level lookup; context data wins; multiple rows imply AND. **Scheduling:** checker only, no timer. **Actions:** sequential, synchronous, bounded HTTP/Resend retries, no exactly-once guarantee.

Identity: Supabase bearer token validated remotely, backend ownership checks when configured. **Local mode:** missing Supabase URL disables those workflow identity checks. **Webhook:** HMAC over raw bytes only when secret configured, shared candidate selection, no deduplication. **Credentials:** Gmail encrypted, GitHub config not encrypted. **UI:** plain CSS, one visible action/condition, local React state, native fetch.

Evidence: 28 backend tests passed; build including TypeScript passed; lint has one warning; live deployment not verified in this preparation. **Main gaps:** production-safe configuration, tenant-aware webhooks, secrets/redaction, accurate timing, richer UI, durable execution if justified.

Practise these five prompts without notes: explain the project in 30 seconds; trace Run now; show polymorphism in code; explain OAuth without using the word password as a token synonym; name one limitation and a test that would expose it.

27. Exercises that prove you understand the code

1. Predict the result of an equals condition over a missing field, and then a not_equals condition over that field. Explain why the outcomes differ.
2. Predict a workflow with two successful actions followed by a failing action: status, action_count, and whether a fourth action runs.
3. Explain why a manual run against a GitHub webhook workflow is usually SKIPPED even if the payload looks like GitHub JSON.
4. Draw the objects produced by a nested AND containing an OR. Point to where factory recursion and evaluation recursion occur.
5. Identify which statement sends SQL and which commits it. Explain how a validation error can roll back a previously flushed parent row.
6. Explain why `repository.full_name` fails in a condition without claiming GitHub sent bad JSON.
7. Identify the two different tokens involved in the GitHub dispatch demonstration and which process uses each.
8. Explain why changing a VITE_ value in a hosting dashboard without rebuilding may leave the old value in the browser bundle.
9. Trace Gmail from a random state string to a hashed state row, authorization code, encrypted tokens, refreshed token, MIME message, and provider result.
10. Describe a test for another user's workflow access. State the expected HTTP status and how to inject the test identity.
11. Explain the consequence of an API-created multi-action workflow being edited in the single-action UI.
12. Propose a safe error/result contract for a new Action, including what it must never put in exceptions or history.

If you can answer these by referring to the actual functions rather than repeating definitions, you are prepared for substantially deeper questioning.

28. Source map, report scope, and further reading

The companion `FlowForge-Line-by-Line.md` is the source-reading appendix. It records a SHA-256 fingerprint for each covered file, preserves line numbering, explains every nonblank line, labels blank lines as formatting, and adds file-specific behavior notes. This makes the explanation auditable against the snapshot instead of relying on an unrelated tutorial.

Coverage includes application Python, backend tests, frontend TS/TSX/CSS, package/configuration files, the HTML entry point, pytest configuration, and the GitHub dispatch YAML. Images/SVG assets are not executable workflow logic; their role and inactive starter status are described. Dependency-generated files, bundles, caches, binary databases, actual environment files, credentials, private logs, and tool configuration are not copied into the guide. Package locks are dependency resolution records rather than handwritten application logic and are not annotated line by line.

Primary repository sources are the current backend/app and frontend/src implementation, test files, README, DEPLOYMENT, agent_docs, MEMORY, and the PRD/technical design/research documents. When older prose conflicts with executable source, this guide labels the discrepancy. Historical rationale is separated from inferred trade-offs.

The linked official documentation throughout the chapters supports language/library mechanisms. It is a learning reference, not proof of FlowForge's deployed state. This report does not inspect secret files, change source behavior, change schemas, contact real recipients, dispatch real provider actions, or configure infrastructure.

The following are useful next reading targets after this project: Python object/data model and exception handling; React state/effect lifecycle; TypeScript narrowing and generics; FastAPI dependencies and response models; SQLAlchemy transactions and relationship cascades; OAuth authorization-code flows; SSRF defenses; and idempotent execution design. Follow each topic back to the concrete FlowForge example that made it necessary.